

Git and GitHub Guide

Quick References, Foundations, Collaboration, and Toolbox

Jonathan Y. Ma

May 12, 2026

Contents

I	Quick Reference Guides	1
1	Git	2
2	Git Classroom	11
II	Detailed Git Guides	15
3	GitHub Foundations	16
4	Git Installation, Configuration, and Core Workflow	49
4.1	Operating System Compatibility	49
4.1.1	Pre-installed Systems	49
4.1.2	Requires Installation	49
4.1.3	Verification	49
4.2	Installing Git on Windows	49
4.2.1	Step 1: Download Git	49
4.2.2	Step 2: Run the Installer	50
4.2.3	Step 3: Configure Installation Options	50
4.2.4	Step 4: Complete Installation	50
4.3	Accessing Git Interfaces	51
4.3.1	Windows	51
4.3.2	macOS	51
4.3.3	Linux	51
4.4	Understanding Git Interfaces	51
4.4.1	Command Line Interfaces	51
4.4.2	Common Command Differences	52
4.5	GUI Alternatives	52
4.5.1	General Recommendation	52
4.6	Basic Git Verification	52
4.6.1	Check Version	52
4.6.2	Explore Basic Commands	52
4.7	Understanding Git Configuration	53
4.8	Configuring User Identity	53
4.8.1	Configure Name	53

4.8.2	Configure Email	53
4.8.3	Why Matching GitHub Email Matters	53
4.9	Understanding Configuration Scope	53
4.9.1	Global Configuration	53
4.9.2	Repository-Specific Configuration	53
4.10	Verifying Configuration	54
4.10.1	View Name	54
4.10.2	View Email	54
4.10.3	View All Configuration	54
4.11	Understanding Git Repositories	54
4.12	Creating a Repository	54
4.12.1	Create Project Directory	54
4.12.2	Initialize Repository	54
4.13	Checking Repository Status	55
4.14	The .git Directory	55
4.15	Creating the First Project File	55
4.16	Understanding File States	56
4.17	Adding Files to Git	56
4.17.1	Check Status	56
4.17.2	Stage the File	56
4.17.3	Verify Staging	56
4.18	Understanding the Staging Area	57
4.18.1	Why the Staging Area Exists	57
4.19	Creating Commits	57
4.19.1	Commit Changes	57
4.19.2	Verify Clean Status	57
4.20	Viewing Commit History	57
4.20.1	Detailed Log	57
4.20.2	Compact Log	57
4.21	Making Additional Changes	57
4.22	Commit Message Best Practices	58
4.23	The Three-Stage Architecture	58
4.24	Understanding git diff	58
4.25	Creating a .gitignore	59
4.26	Why .gitignore Matters	59
4.27	Common .gitignore Patterns	59
4.27.1	Python	59
4.27.2	Node.js	59
4.27.3	LaTeX	59
4.28	Branching Concepts	60
4.29	Creating Branches	60
4.29.1	Create Branch	60
4.29.2	Switch Branch	60
4.29.3	Create and Switch Simultaneously	60
4.30	Branch Visualization	60

4.31	Merging Branches	60
4.32	Deleting Branches	61
4.33	Understanding Git Stash	61
4.33.1	Create Stash	61
4.33.2	View Stashes	61
4.33.3	Restore Stash	61
4.34	Local vs Remote Repositories	61
4.34.1	Local Repository	61
4.34.2	Remote Repository	61
4.35	Connecting to GitHub	62
4.35.1	Add Remote	62
4.35.2	Verify Remote	62
4.36	Pushing to GitHub	62
4.36.1	First Push	62
4.36.2	Subsequent Pushes	62
4.37	Pulling Changes	62
4.37.1	Fetch Remote Updates	62
4.37.2	Pull Remote Changes	62
4.37.3	Difference Between Fetch and Pull	62
4.38	GitHub Pages	62
4.38.1	Enable GitHub Pages	63
4.38.2	Resulting URL	63
4.39	GitHub Desktop	63
4.40	Recommended Beginner Workflow	63
4.41	Essential Git Commands Summary	64
5	GitHub Collaboration	65
5.1	What Are GitHub Issues?	65
5.1.1	Historical Context	65
5.1.2	Modern Usage	65
5.1.3	Community Adoption	66
5.2	Best Practices for Writing Issues	66
5.2.1	Software Projects	66
5.2.2	Non-Software Projects	66
5.3	Linking to Specific Code or Content	67
5.3.1	Creating Line References	67
5.3.2	Reference Formats	67
5.3.3	Permalinks	67
5.4	Issues Beyond Software Development	67
5.4.1	Academic Applications	67
5.4.2	Research Examples	68
5.5	Issue Categories	68
5.5.1	Bugs	68
5.5.2	Features and Enhancements	68
5.5.3	General Tasks	68

5.6	Applying Labels to Issues	69
5.6.1	Accessing Labels	69
5.6.2	Common Labels	69
5.6.3	Viewing Labels	69
5.7	Filtering Issues	69
5.7.1	Label-Based Filtering	69
5.7.2	Organizational Benefits	69
5.8	Customizing Labels	70
5.8.1	Managing Labels	70
5.8.2	Academic Label Examples	70
5.8.3	Software Label Examples	70
5.9	Advanced Label Usage	70
5.9.1	Duplicate Issues	70
5.9.2	Won't Fix	71
5.9.3	Good First Issue	71
5.10	Assigning Issues	71
5.10.1	Assigning Team Members	71
5.10.2	Benefits of Assignment	71
5.11	Multiple Assignees	71
5.12	Academic Applications of Assignment	72
5.12.1	Research Teams	72
5.12.2	Course Development	72
5.13	GitHub Flow Branch Strategy	72
5.13.1	Why Use Branches?	72
5.13.2	GitHub Flow Process	72
5.14	Creating Branches	73
5.14.1	Check Current Branch	73
5.14.2	Create and Switch Branch	73
5.14.3	Verify Branch	73
5.15	Branch Naming Conventions	73
5.15.1	Common Naming Patterns	73
5.15.2	Including Issue Numbers	73
5.16	Working in Branches	73
5.16.1	Development Process	73
5.16.2	Verification	74
5.17	Testing and Validation	74
5.18	Understanding Pull Requests	74
5.18.1	Definition	74
5.18.2	Why Pull Requests Matter	74
5.19	Creating Pull Requests	74
5.19.1	Basic Workflow	74
5.19.2	Branch Selection	75
5.20	Writing Pull Request Descriptions	75
5.20.1	Issue References	75
5.21	Code Review	75

5.21.1	Purpose of Review	75
5.21.2	Review Process	75
5.22	Pull Request Interface	76
5.22.1	Files Changed	76
5.23	Task Lists in Issues	76
5.23.1	Markdown Syntax	76
5.23.2	Interactive Features	76
5.24	Sub-Issues	76
5.25	Progress Tracking	76
5.26	GitHub Codespaces	77
5.26.1	Definition	77
5.26.2	Capabilities	77
5.27	Launching Codespaces	77
5.27.1	Workflow	77
5.27.2	Environment Setup	77
5.28	Branching in Codespaces	77
5.29	Codespaces Advantages	78
5.30	Codespaces Considerations	78
5.31	Reviewing Pull Requests	78
5.31.1	Review Workflow	78
5.31.2	Review Types	78
5.32	Line-Level Feedback	78
5.33	AI-Assisted Review	79
5.34	Updating Pull Requests	79
5.35	Resolving Conversations	79
5.36	Approving Pull Requests	79
5.37	Merging Pull Requests	80
5.37.1	Merge Process	80
5.37.2	Post-Merge Actions	80
5.38	Merge Conflicts	80
5.38.1	What Causes Conflicts?	80
5.38.2	Conflict Indicators	80
5.39	Conflict Resolution	80
5.39.1	Conflict Markers	80
5.39.2	Resolution Steps	81
5.40	Preventing Conflicts	81
5.41	Understanding Milestones	81
5.41.1	Purpose	81
5.42	Creating Milestones	81
5.43	Milestone Progress	82
5.44	Project Boards	82
5.44.1	Kanban Workflow	82
5.45	Board Automation	82
5.46	Discussion Forums	82
5.46.1	Discussions vs. Issues	82

5.47	Discussion Categories	83
5.48	GitHub Wikis	83
5.48.1	Purpose	83
5.48.2	README vs. Wiki	83
5.49	Wiki Organization	83
5.50	Issue Templates	84
5.50.1	Benefits	84
5.51	Bug Report Template Example	84
5.52	Feature Request Template Example	84
5.53	Markdown vs YAML Templates	84
5.53.1	Markdown Templates	84
5.53.2	YAML Forms	85
5.54	Fork-Based Collaboration	85
5.54.1	Why Forks Exist	85
5.55	Creating Forks	85
5.55.1	Fork Relationship	85
5.56	Fork Workflow	85
5.56.1	Contribution Process	85
5.57	Cross-Repository Pull Requests	85
5.58	Open Source Contribution Etiquette	86
5.59	Complete GitHub Collaboration Workflow	86
5.60	Collaboration Beyond Software	86
6	GitHub Education	87
6.1	What is GitHub Classroom?	87
6.1.1	Platform Access	87
6.1.2	Core Problem Solved	87
6.2	Alternative Approaches	88
6.2.1	Manual Repository Management	88
6.2.2	Student Self-Setup	88
6.2.3	API-Based Automation	88
6.2.4	GitHub Classroom Advantages	88
6.3	Platform Evolution and Capabilities	88
6.3.1	Origin and Development	88
6.3.2	Current Feature Set	89
6.3.3	Future Considerations	89
6.4	Organization Requirements and Relationships	89
6.4.1	Required Setup	89
6.4.2	Organization Strategies	89
6.5	Creating Educational Organizations	90
6.5.1	Step 1: Organization Creation	90
6.5.2	Naming Convention Examples	90
6.6	Organization Configuration Considerations	91
6.6.1	Free vs. Paid Organization	91
6.6.2	Funding Options	91

6.6.3	Privacy and Access Control	91
6.7	Classroom Creation Process	91
6.7.1	Step 1: Access Classroom Creation	91
6.7.2	Step 2: Organization Selection	92
6.7.3	Step 3: Classroom Naming	92
6.8	Initial Configuration Options	92
6.8.1	Teaching Staff Configuration	92
6.8.2	Student Roster Management	92
6.9	Classroom Interface Overview	92
6.9.1	Main Interface Tabs	92
6.10	Repository Creation Requirements	93
6.10.1	Organizational Requirement	93
6.10.2	Repository Visibility	93
6.11	Template Repository Setup	93
6.11.1	Step 1: Repository Creation	93
6.11.2	Step 2: Add Assignment Content	94
6.11.3	Step 3: Configure Template Settings	94
6.12	Template Repository Features	94
6.12.1	Key Benefits	94
6.13	Assignment Design Considerations	94
6.13.1	Assignment Instructions	94
6.13.2	Starter Materials	95
6.14	Assignment Creation Workflow	95
6.14.1	Step 1: Access Assignment Creation	95
6.14.2	Step 2: Basic Assignment Configuration	95
6.14.3	Step 3: Assignment Type Selection	95
6.15	Repository Configuration Options	96
6.15.1	Template Repository Selection	96
6.15.2	Student Repository Visibility	96
6.15.3	Access Control Settings	96
6.16	Advanced Assignment Options	96
6.16.1	Branch Management	96
6.16.2	Specialized Features	97
6.17	Assignment Dashboard and Management	97
6.17.1	Assignment Dashboard Features	97
6.17.2	Distributing Assignment Invites	97
6.18	Student Acceptance Workflow	97
6.18.1	Step 1: Assignment Access	97
6.18.2	Step 2: Repository Naming Convention	97
6.18.3	Step 3: Assignment Acceptance	98
6.19	Repository Organization and Permissions	98
6.19.1	Repository Characteristics	98
6.19.2	Student Permissions	98
6.19.3	Instructor Permissions	98
6.20	Classroom Dashboard Monitoring	98

6.20.1	Assignment Tracking Features	98
6.20.2	Administrative Tools	98
6.21	Student Workflow Overview	99
6.21.1	Development Process	99
6.21.2	Submission Recognition	99
6.22	Practical Submission Example	99
6.22.1	Step 1: Assignment Modification	99
6.22.2	Step 2: Committing and Submission	99
6.22.3	Step 3: Submission Confirmation	99
6.23	Submission Tracking Limitations	99
6.23.1	Push-Equals-Submission Model	99
6.23.2	Potential Issues	100
6.24	Alternative Submission Strategies	100
6.24.1	Instructor Communication	100
6.24.2	Workflow Adaptations	100
6.25	Gradescope Assignment Configuration	100
6.25.1	Step 1: Create Programming Assignment	100
6.25.2	Step 2: Enable GitHub Submission Method	100
6.26	GitHub Account Integration	100
6.26.1	Instructor Setup	100
6.26.2	Student Setup	100
6.27	Organizational Access Configuration	101
6.27.1	Critical Requirement	101
6.27.2	Configuration Steps	101
6.28	Student Submission Workflow	101
6.28.1	Repository Selection	101
6.28.2	Submission	101
6.29	Workflow Limitations and Considerations	101
6.29.1	Manual Submission Requirement	101
6.29.2	Integration Benefits	101

Part I

Quick Reference Guides

Chapter 1

Git

Purpose

This page is a compact Git reference for the most common tasks in this course. Use it when you remember the concept but do not want to rewatch an entire lesson.

Scope

Local Git commands, basic branching, undo operations, and syncing with GitHub.

Terminal Note

- **macOS:** Use the built-in Terminal app for installation, setup, and first-time verification.
- **VS Code terminal:** Fine for daily Git work after your tools are already installed and working.
- **Windows:** Use Git Bash unless your lesson specifically says otherwise.

Core Workflow

```
git status
git add .
git commit -m "Describe your change"
git push
```

Starting a Repository

New Local Repository

```
git init
```

Clone an Existing GitHub Repository

```
git clone https://github.com/username/repository-name.git
```

Connect a Local Repository to GitHub

```
git remote add origin https://github.com/yourusername/repository-name.git  
git push -u origin main
```

Use your repository's default branch name, which is often `main` and may be `master` in some repositories.

Checking What Changed

Use this when you are not sure what you changed, what is staged, or what was committed recently.

```
git status  
git diff  
git diff --staged  
git log --oneline
```

- `git status`: overall summary
- `git diff`: unstaged changes
- `git diff -staged`: staged changes
- `git log -oneline`: compact commit history

Staging and Committing

Stage One File

```
git add filename.txt
```

Stage Everything in the Current Directory

```
git add .
```

Create a Commit

```
git commit -m "Add score display to game UI"
```

Commit Message Pattern

- Start with a verb: Add, Fix, Update, Remove, Refactor
- Describe one logical change
- Keep it short but specific

Branching

Create and Switch to a New Branch

```
git switch -c feature-branch
```

Switch Branches

```
git switch main
```

or

```
git switch master
```

List Branches

```
git branch
```

Merge a Branch into Your Current Branch

```
git merge feature-branch
```

Typical Pattern

```
git switch -c feature-branch  
  
# make changes  
  
git add .  
git commit -m "Add new feature"
```

```
git switch main  
git merge feature-branch
```

Undo and Recovery

Discard Unstaged Changes in One File

```
git restore filename.txt
```

Unstage a File Without Losing Changes

```
git restore --staged filename.txt
```

Restore Everything in the Working Directory

```
git restore .
```

Temporarily Save Unfinished Work

```
git stash  
git stash pop
```

Important Warning

Do not use `git reset -hard` unless you are sure you want to discard uncommitted work and rewrite your current branch position.

Syncing with GitHub

Push Your Commits

```
git push
```

Download Remote Changes and Merge Them

```
git pull
```

Check for Remote Updates Without Merging

```
git fetch
```

Recommended Pattern Before Starting Work

```
git status
git pull
```

Recommended Pattern When Finishing Work

```
git status
git add .
git commit -m "Describe your change"
git push
```

Common Situations

Situation	Command
I changed a file and want Git to notice it	<code>git status</code>
I want to include the change in my next commit	<code>git add filename.txt</code>
I finished a logical unit of work	<code>git commit -m "Describe the completed change"</code>
I want my GitHub repository to match my local work	<code>git push</code>
I need the latest changes from GitHub	<code>git pull</code>
I want to save work quickly and switch tasks	<code>git stash</code>

Minimal Command Set

If you only remember a few commands, remember these:

```
git status
git add .
git commit -m "message"
git push
git pull
git switch -c branch-name
git merge branch-name
```

```
git restore filename.txt
```

Typical Solo Workflow

1. Create a repository on GitHub
2. Clone it locally or connect your local repository to it
3. Make changes on your computer
4. Commit locally with Git
5. Push to GitHub
6. Refresh the repository page to confirm your changes are there

Typical Collaboration Workflow

1. Create or identify an issue
2. Create a branch for the work
3. Make changes and push the branch
4. Open a pull request
5. Request review
6. Address feedback
7. Merge after approval

Issues

Use issues for:

- Bug reports
- Feature requests
- Assignment planning
- Task tracking

Good Issue Contents

- Clear title
- Short description of the problem or goal
- Relevant context
- Labels if your project uses them
- Assignee if ownership is known

Pull Requests

Open a Pull Request When

- You want your branch reviewed before merging
- You are contributing to a team project
- You want a record of discussion about a change

Good Pull Request Contents

- What changed
- Why it changed
- Anything reviewers should test or pay attention to

Review Outcomes

- **Approve:** ready to merge
- **Comment:** discussion or suggestions
- **Request changes:** needs more work before merging

Branches on GitHub

- The default branch is often `main` and may be `master` in some repositories
- Feature branches keep unfinished work separate
- Branches make pull requests possible

Use a Branch When

- You are adding a feature
- You are fixing a bug
- You want to experiment safely

Collaborators and Access

Collaborators

People you invite to work in a repository.

Useful for small projects or direct sharing.

Organizations

Shared spaces for teams, labs, courses, or departments.

Better for managing multiple repositories and multiple people.

Teams

Groups inside an organization used for permissions and coordination.

GitHub Classroom

Use GitHub Classroom For

- Distributing assignments
- Creating one repository per student or per team
- Managing roster-related workflows

Instructor Flow

1. Create a classroom
2. Create an assignment
3. Share the invite link
4. Monitor acceptance and submissions

Student Flow

1. Open the invite link
2. Accept the assignment
3. Get a personal or team repository
4. Do the work and push commits

Codespaces, Desktop, and Web Editing

GitHub Desktop

Best for users who want a visual Git workflow on their own computer.

Codespaces

Cloud development environment running in the browser.

Useful when you want a consistent setup without local configuration.

Web Editing on GitHub

Good for small quick edits.

Not ideal for larger development tasks.

Choose Based on the Task

Task	Recommended Tool
Quick edit	GitHub web interface
Visual local workflow	GitHub Desktop
Full cloud environment	Codespaces

Chapter 2

Git Classroom

For Instructors: Basic Workflow

1. Create or select a GitHub organization
2. Create a GitHub Classroom linked to that organization
3. Create an assignment
4. Select the template repository or starter materials
5. Copy the invite link
6. Share the invite link with students
7. Monitor which students accepted the assignment
8. Review student repositories and submissions

For Instructors: What You Need Before You Start

- A GitHub account
- A GitHub organization for the course or teaching context
- Access to GitHub Classroom
- A template repository if you want students to start from common files

For Instructors: Creating an Assignment

Good Assignment Setup Pattern

1. Write instructions in the template repository README

2. Include any starter code, files, or data
3. Create the classroom assignment from that template
4. Test the invite link before sending it to students

For Instructors: Invite Links and Student Acceptance

Invite Links

- Each assignment has an invite link
- Students use that link to accept the assignment
- The acceptance process creates a repository for the student or team

Best Practice

- Post the invite link in Canvas, email, or course announcements
- Tell students they must accept the assignment before they can begin work in the correct repository

Important Distinction

Sending the link is not the same as a student accepting the assignment.
Acceptance creates the repository.

For Instructors: Reviewing Student Work

Common Ways to Review Submissions

- Open student repositories directly on GitHub
- Check commit history
- Review files and changes
- Clone repositories locally if deeper inspection is needed

Useful Signals

- Whether the student accepted the assignment
- Whether the repository has commits
- When commits were pushed
- Whether required files are present

For Instructors: Rosters, Identity Mapping, and Canvas

Roster Management

You may manage students manually or through supported LMS-related workflows.

Identity Mapping

Identity mapping connects student names with GitHub usernames.

Canvas Reminder

Canvas integration is mainly about roster-related workflows.

Grades still need to be recorded in Canvas separately.

Do not assume GitHub Classroom automatically sends grades back to Canvas.

For Students: Basic Workflow

1. Open the assignment invite link
2. Accept the assignment
3. Wait for your repository to be created
4. Open the repository on GitHub
5. Clone it locally, open it in Codespaces, or edit in the browser if appropriate
6. Do the assignment work
7. Commit your changes
8. Push your commits to GitHub

For Students: Important Reminders

- Accepting the assignment creates the repository, but it does not complete the work for you
- Working locally is not enough by itself; you must push your commits if your instructor expects the work on GitHub
- Use the repository created by GitHub Classroom, not a separate personal repository
- Check the assignment README for instructions before you start changing files

For Students: If You Are Working in a Team Assignment

- Make sure you join the correct team during the acceptance process
- Confirm that your teammates can all see the same repository
- Coordinate who works on which parts of the assignment
- Pull before starting work if teammates may have already pushed updates

Part II

Detailed Git Guides

Chapter 3

GitHub Foundations

Creating a GitHub Account

Step 1: Navigate to GitHub

Go to <https://github.com> and click “Sign up for GitHub.”

Step 2: Account Registration

Email Address

- Enter your preferred email address.
- You can use any email address: personal, school, or alias.
- Consider using a professional email you will maintain long-term.

Password

Create a strong, secure password.

Username Selection

- Choose a professional username.
- Usernames must be globally unique across all GitHub users.
- Consider using variations of your name if your preferred choice is taken.
- Example approaches: `firstname-lastname`, `firstnamelastname`, or `firstname_lastname`.

Account Preferences

- Select your country or region.
- Choose whether to receive product updates. This is optional.

Step 3: Email Verification

1. Click “Create account.”
2. Check your email for a verification code.
3. Enter the verification code when prompted.
4. Complete account creation.

Professional Username Tips

Choose a username you will be comfortable using professionally. Many employers and collaborators will see this username, so avoid overly casual or temporary-sounding names.

Updating Your GitHub Profile

Step 1: Access Your Profile

1. Once logged in, click on your profile picture in the top right.
2. Select “Settings” from the dropdown menu.

Step 2: Basic Profile Information

- **Profile Picture:** Upload a professional photo or avatar.
- **Display Name:** Add your full name.
- **Bio:** Consider adding a brief description. This is optional.
- **Location:** Add your location if desired. This is optional.

Step 3: Save Changes

Review your profile information and save any changes made.

Your new GitHub account will start empty: no repositories, no activity, and minimal profile information. This is normal and expected. The GitHub interface will provide getting started suggestions and tutorials for new users.

Optional: school Email Aliases

For faculty and staff who want to create demonstration accounts or test different user perspectives, school offers email alias services through <https://my.school.edu>. This allows you to:

- Create up to three email aliases.

- Use aliases for testing student workflows.
- Demonstrate different user roles in GitHub Classroom.
- Maintain separation between personal and demonstration accounts.

To access this service:

1. Go to <https://my.school.edu>.
2. Search for “Email Alias.”
3. Follow the setup instructions provided.

Note: This feature may not be available to all user types. Check with IT services for your specific access level.

GitHub Education Benefits

GitHub Education provides enhanced features for students, faculty, and educational institutions. The benefits structure works in tiers:

- **Free Account:** Basic GitHub features.
- **Education Benefits:** Access to additional GitHub Education features and offers for eligible students and educators.
- **Enterprise Access:** Additional enterprise features through school Single Sign-On.

Pre-Application Requirements

Step 1: Access Security Settings

1. Go to <https://github.com> and sign into your account.
2. Click on your profile image in the top right corner and select “Settings.”
3. Navigate to “Password and authentication” on the left sidebar.
4. Under “Password and authentication,” click “Enable two-factor authentication.”

Step 2: Choose Authentication Method

Option A: Authenticator App Recommended

- Install the GitHub Mobile app.
- Follow setup instructions for push notifications.

Option B: SMS

Use your phone number for text message codes.

Step 3: Save Recovery Codes

Download and securely store backup codes.

Step 4: Navigate to Billing Settings

Go to Settings → Billing and licensing → Payment information.

Add a billing address and payment method.

Note: You will not be charged for education benefits.

Applying for Education Benefits**Step 1: Access Application**

Go to Settings → Billing and licensing → Education benefits.

Click “Start an application.”

Step 2: Complete Application**Role Selection**

- Faculty or staff: Select “Teacher.”
- Students: Select “Student.”

Institution

Type “Johns school University” and select it from the dropdown.

Location Sharing

Allow location sharing when prompted.

If issues occur, try a different browser or disable VPN.

Academic Verification

Choose one:

- Upload a photo of your school ID card.
- Upload a screenshot of your <https://my.schoolu.edu> profile showing affiliation.

Step 3: Submit and Wait

- Applications usually process in 1–3 days.
- You will receive an email notification when approved or rejected.
- Check spam and junk folders.

Using Education Benefits

Step 1: Navigate to Dashboard

Direct URLs:

- Faculty: <https://education.github.com/globalcampus/teacher>
- Students: <https://education.github.com/globalcampus/student>

Step 2: Explore Available Benefits

- GitHub Team upgrades for organizations.
- Eligible users may receive access to GitHub Copilot education benefits, subject to GitHub’s current program terms.
- Educational resources.
- Additional tools and discounts.

Creating Course Organizations for Faculty

Step 1: Create Organization

1. From the GitHub main page, click “+”.
2. Select “New organization.”
3. Choose “Create a free organization.”
4. Choose an organization name, such as `schoolu-course-name`.
5. Set organization visibility to “Public.” You can change this later once upgraded to a Team account.
6. Complete setup with your email.

Step 2: Upgrade to GitHub Team

1. From the Education Dashboard, click “Upgrade to GitHub Team.”
2. Select your course organization among the options, if any.
3. Click “Upgrade.”
4. Confirmation email arrives within an hour, and often much sooner.

Benefits of Team Upgrade

- Unlimited private repositories.
- Unlimited collaborators.
- Access to GitHub Classroom features.
- Access to GitHub Codespaces and Copilot Pro.

Annual Renewal: Education benefits require renewal every 1–2 years. Monitor email notifications for renewal reminders.

Understanding the GitHub Access Progression

The Three-Layer Approach

1. **Personal GitHub Account:** Your individual presence on <https://github.com>.
2. **Education Benefits:** Enhanced features through GitHub Global Campus.
3. **Enterprise Access:** Institutional integration with school resources.

Each layer builds upon the previous one, providing additional capabilities while preserving previous functionality.

Requesting Organization Access

Step 1: Access Request Form

From the school Enterprise page, click “request form” to access organizational resources.

Step 2: Complete Request Information

Organization Options

- **Use Existing:** If you have already created an organization through education benefits.
- **Create New:** Request a new organization for your course or project.

Required Information

- Organization name, existing or proposed.
- Your GitHub username, not display name.
- Brief description of intended use.
- Organization owner information.

Step 3: Submit and Track

- Submit the completed form.
- Await email confirmation of approval.
- Organizations are typically approved within 1–2 business days.

Existing vs. New Organizations: If you created an organization through education benefits, request to add it to school Enterprise for additional institutional features and support.

If you want to be invited to an organization that is part of the Enterprise, ask the organization owner for an invitation. If you do not know who the owner is, reach out to the OSPO at ospo@schoolu.edu.

Accessing school GitHub Enterprise

Step 1: Navigate to school SSO

Access school GitHub Enterprise using:

```
https://github.com/enterprises/johns-school-university/sso
```

Step 2: Initiate Single Sign-On

1. Click “Continue” when prompted about Single Sign-On.
2. You will be redirected to the school authentication system.
3. Enter your schoolED credentials as you would for any school system.
4. Complete any additional authentication steps, such as 2FA if enabled.

Step 3: Verify Enterprise Access

After successful authentication:

- The page header shows “Johns school University.”
- You maintain access to your personal <https://github.com> account.
- You now have access to school organizational resources.

Navigating the school Enterprise Interface

Step 1: Access Enterprise Dashboard

The school Enterprise landing page provides:

- Welcome message and getting started information.
- List of existing school organizations.
- Links to request new organizational access.

Step 2: Understand Available Organizations

- View organizations you already have access to.
- See public school organizations.
- Identify opportunities for collaboration.

Step 3: Bookmark Access Methods

Method 1: Direct URL

Bookmark:

```
https://github.com/enterprises/johns-school-university
```

Method 2: GitHub Menu Access

1. From any GitHub page, click your avatar.
2. Select “Your enterprises.”
3. Click “Johns school University.”

Enterprise vs. Personal Account Functionality

Preserved Personal Features

- All personal repositories remain accessible.
- Individual education benefits continue.
- Personal profile and contributions remain unchanged.
- Existing collaborations and organizations are maintained.

Added Enterprise Benefits

- Access to school institutional organizations.
- Enhanced security and compliance features.
- Integration with school identity systems.
- Institutional support and resources.
- Advanced administrative capabilities.
- Unlimited private repositories.
- Ability to create “internal” repositories that are limited to school affiliates.
- Ability to request paid add-ons such as large runners.

Seamless Integration

- Switch between personal and enterprise contexts.
- Use a single login for both personal and institutional work.
- Access unified notification and activity feeds.

Processing Time: Organization requests through school Enterprise typically process within 1–2 business days. You will receive email confirmation when access is granted.

Planning Tasks

- **Identify Use Cases:** List potential courses, projects, or collaborations that could benefit from school organizational access.
- **Organization Strategy:** Decide whether to request access to existing school organizations, transfer education-benefit organizations to school Enterprise, or create new organizations specifically for school work.

- **Integration Planning:** Consider how enterprise access fits with your course management needs, research collaboration requirements, and student interaction workflows.
- **Account Separation:** Maintain clear separation between personal projects and institutional work to ensure appropriate privacy and access control.

Accessing Repository Creation

Method 1: From Dashboard

Log in to GitHub at <https://github.com>. From your dashboard, click the green “New” button.

Method 2: From Repositories Tab

Click on your profile, go to the “Repositories” tab, and click “New.”

Method 3: Direct URL

Navigate directly to:

```
https://github.com/new
```

All methods lead to the same repository creation page.

Repository Configuration

Step 1: Repository Name

Enter a repository name, such as:

```
lecture-notes
```

Important naming rules:

- Must be unique within your account or organization.
- Can only contain ASCII letters, digits, dots, dashes, and underscores.
- Spaces are automatically converted to dashes.
- Not required to be globally unique across all GitHub.

Step 2: Owner Selection

- **Personal Account:** Only your username appears.
- **Organizations:** If you belong to organizations, you can choose which account or organization owns the repository.

Step 3: Description Optional

Add a brief description. For example:

Lecture notes for data structures.

Step 4: Visibility Setting

Choose repository visibility:

- **Public:** Anyone can see this repository.
- **Private:** Only you and collaborators you choose can see this repository.

Select **Public** for sharing lecture materials with students.

Step 5: Initialization Options

For now, leave all initialization options unchecked:

- Add a README file.
- Add `.gitignore`.
- Choose a license.

These will be explored in future lessons.

Repository Naming Best Practices

- Use descriptive names that indicate the project's purpose.
- Use lowercase letters and hyphens for consistency.
- Avoid special characters and spaces.
- Consider including course codes or years for academic repositories.

Step 6: Create Repository

1. Review all settings.
2. Click “Create repository.”
3. GitHub will redirect you to your new repository page.

Repository Name Permanence: While you can rename repositories later, it is best practice to choose meaningful names from the start. Renaming can break existing links and references.

Understanding the New Repository Interface

After creating your repository, you will see a page with multiple setup options and information. This interface can appear overwhelming initially, but it provides several pathways for getting started.

Key areas you will see:

- Quick setup instructions.
- Options for adding files.
- Clone and download information.
- Git command examples.

What to focus on: For now, familiarize yourself with the layout. File uploading and Git commands will be explored in subsequent lessons.

Repository Ownership and Organizations

Personal Repositories

- Owned by your individual GitHub account.
- You have full control over settings and access.
- Suitable for personal projects and individual coursework.

Organization Repositories

- Owned by a GitHub organization.
- Multiple members can have admin access.
- Ideal for course materials, team projects, and collaborative work.

Organization repository creation will be demonstrated in future lessons.

Accessing the File Upload Interface

From Your Empty Repository

1. On your repository's main page, look for "uploading an existing file."
2. Click the link to access the upload interface.

Direct URL Method

Navigate to:

```
https://github.com/[username]/[repository-name]/upload
```

Replace [username] with your GitHub username and [repository-name] with your repository name.

Uploading Files

Step 1: Select Files

Method A: Drag and Drop

1. Open file explorer or Finder on your computer.
2. Select the files you want to upload.
3. Drag them into the upload area on GitHub.

Method B: Browse to Select

1. Click “choose your files.”
2. Browse and select files from your computer.
3. Click “Open” to add them to the upload queue.

Step 2: Review Selected Files

- All selected files appear in the upload area.
- To remove unwanted files, click the “X” button next to each file.
- Verify you have the correct files selected.

Step 3: Add Commit Message

GitHub provides a default commit message:

```
Add files via upload
```

You can modify this message to be more descriptive:

```
Add lecture slides for data structures course
```

Step 4: Commit Files

Click “Commit changes.” GitHub will process and add your files to the repository. You will be redirected to your repository’s main page.

File Selection Strategy: Start with a few files to get familiar with the process. You can always upload additional files later using the same method.

Understanding Public Repository Access

Public Repository Visibility

- Anyone can view the repository and its contents.
- No GitHub account is required to view files.
- Others cannot make changes without explicit permission from the repository owner.
- Others can download files individually or as a complete package.

Public repositories are ideal for:

- Educational materials you want to share with students.
- Open source projects.
- Portfolio showcases.
- Documentation that should be publicly accessible.

To verify public access works correctly:

- **Logged-in Users:** Other GitHub users can view your repository by visiting the URL.
- **Non-logged-in Users:** People without GitHub accounts can still view and download files.
- **Repository Owner:** You have full access to view, edit, and manage the repository.

All user types see the same content, but only the owner and invited collaborators can make changes.

Downloading Files from Repositories

Individual File Download

1. Click on any file name in the repository.
2. For viewable files, such as text or images, GitHub displays the content.
3. For non-viewable files, such as PowerPoint or Word documents, click “Download.”
4. The file downloads to your computer’s default download location.

Bulk Download Entire Repository

1. Click the green “Code” button on the repository main page.
2. Select “Download ZIP” from the dropdown menu.
3. A ZIP file containing all repository contents downloads.
4. Extract the ZIP file to access all files locally.

File Type Considerations

GitHub can store any file type, but some files work better than others.

Good for GitHub

- Text files, such as `.txt` and `.md`.
- Code files, such as `.py`, `.java`, `.html`, and `.css`.
- Data files such as `.csv`.
- Images such as `.png`, `.jpg`, and `.gif`.
- Configuration files.

Less Ideal for GitHub

- Binary files, such as PowerPoint, Word, and PDF files.
- Large media files, such as videos or large images.
- Compressed files, such as `.zip` and `.rar`.

Reason: GitHub excels at tracking changes in text-based files but cannot effectively show differences in binary files.

Preview Limitations: GitHub cannot render previews for certain file types like PowerPoint presentations or Word documents. Users can still download these files, but they will not be able to preview the content directly in the browser.

The Purpose and Power of README Files

README files serve multiple purposes:

- **Project Description:** Explain what your repository contains.
- **Usage Instructions:** Help visitors understand how to use your files.

- **Navigation Aid:** Provide links to important files or sections.
- **Professional Presentation:** Create a polished, organized appearance.

The `.md` extension stands for Markdown, a lightweight markup language that allows you to format text using simple, readable syntax.

Creating Your First README

Step 1: Access README Creation

From your repository's main page:

1. Look for the suggestion "Add a README."
2. Click "Add a README."
3. GitHub opens a web-based text editor.

Step 2: Add Basic Content

Start with a title, add a brief description, and include necessary context for visitors.

Example content:

Lecture Slides for EN.601.226 Data Structures

Step 3: Commit Your README

1. Scroll down to the commit section.
2. GitHub provides a default commit message: `Add README`.
3. Click "Commit changes" to save your README.

Understanding Markdown Basics

Markdown uses simple text symbols to create formatted content. The GitHub editor provides two tabs:

- **Edit:** Where you write your Markdown code.
- **Preview:** Shows how your formatted text will appear.

Markdown Example

```
# Lecture Slides for EN.601.226 Data Structures
```

This repository contains lecture slides in PowerPoint format from Fall 2024.

- Lecture 1: Course Overview (01.pptx)
- Lecture 2: Java Basics (02.pptx)
- [Lecture 1: Course Overview](01.pptx)
- [Lecture 2: Java Basics](02.pptx)

Link syntax:

```
[Display Text](filename or URL)
```

Advanced Markdown Features

Table Structure

```
| Week | Lecture | Topic | File |
| ---- | - - - - | - - - - - - - - - - | - - - - - - - - - - |
| 1 | 1 | Course Overview | [01.pptx](01.pptx) |
| 1 | 2 | Java Basics | [02.pptx](02.pptx) |
```

Markdown Quick Reference

```
# Header 1
## Header 2
### Header 3

**bold text**
*italic text*

- bullet point
1. numbered list

[link text](URL)
```

Tables use | to separate columns.

Understanding Edit Permissions

- **Repository Owner:** You have full edit access to your README and all files.

- **Public Visitors:** Others can see an “Edit” button, but clicking it prompts them to fork the repository, make changes in their copy, and submit a pull request.

This system protects repositories while still allowing community contributions through a review process.

Commit Messages and Change Tracking

Every time you save changes in GitHub, you create a commit. Think of commits as:

- Save points in your project’s history.
- Documentation of what changed and when.
- A way to track contributions and progress.

Good commit message practices:

- Be descriptive: `Add course schedule table` instead of `Update README`.
- Use present tense: `Add` instead of `Added`.
- Keep messages concise but informative.

Accessing GitHub’s Markdown Guide

1. When editing any file in GitHub, look for “Styling with Markdown is supported.”
2. Click this link to open GitHub’s Markdown guide.
3. Explore text formatting, code blocks, images, tables, task lists, and checkboxes.
4. Bookmark the guide for future README improvements.

Viewing Recent Activity

Step 1: Locate Recent Activity Section

On your repository’s main page, look at the top area above your file listing. You will see:

- Latest commit information.
- Author name.
- Commit message.
- Timestamp.
- Commit identifier.

Step 2: Understand Collaborative Context

If you are working alone, you will only see your username. In team projects, you will see different contributors' names as they make changes.

Accessing Full Repository History

1. Find the clock icon or text showing the number of commits.
2. Click the commits button to see a chronological list of all changes.
3. Select any commit to examine details.

Each history entry shows:

- Commit message.
- Author.
- Date and time.
- Commit hash.

Understanding Diff Views

A diff shows exactly what changed between two versions of a file.

- Red lines with minus signs show content that was removed.
- Green lines with plus signs show content that was added.
- White or gray lines show unchanged content for context.

Reading Diffs Effectively

- Focus on green added sections and red removed sections.
- Use context lines to understand where changes occurred.
- Large additions often indicate new features or content.
- Large deletions may indicate cleanup or removal of outdated information.

The Value of Change Tracking

For Software Development

- Debug issues by seeing when problems were introduced.
- Understand the evolution of code features.
- Credit team members for their contributions.
- Revert problematic changes when necessary.

For Academic Projects

- Track progress on research papers or documentation.
- See the evolution of ideas and analysis.
- Collaborate while maintaining accountability.
- Create a portfolio demonstrating your development process.

For Teaching Materials

- Update content while preserving previous versions.
- Track improvements to curriculum over time.
- Share development process with teaching assistants.
- Maintain course materials across multiple semesters.

Version Control vs. File Backup: File backup saves copies of files at different times. Version control tracks specific changes, who made them, and why, allowing for sophisticated collaboration and project management.

Accessing Repository Settings

1. From the repository main page, locate the horizontal tab bar.
2. Click “Settings” on the far right.
3. If you do not see Settings, you likely do not have administrative access.

Common tabs include Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings.

Renaming Your Repository

1. In Settings, find the “Repository name” section.
2. Enter a new name, such as `226-lecture-notes`.
3. Click “Rename.”
4. Update saved bookmarks or shared links.

GitHub automatically redirects old URLs to the new repository name until you create a new repository with the old name.

Managing Repository Visibility

Repository visibility controls who can see and access your repository.

- **Public:** Anyone on the internet can view the repository.
- **Private:** Only you and people you explicitly invite can access the repository.

Changing Visibility

1. Scroll to the bottom of Settings.
2. Find the “Danger Zone.”
3. Click “Change visibility.”
4. Select Public or Private.
5. Complete GitHub’s confirmation process.

Visibility Change Implications

Making Public to Private

- All public access immediately stops.
- Shared links become inaccessible to non-collaborators.
- Good for protecting sensitive or incomplete work.

Making Private to Public

- Repository becomes visible to everyone immediately.
- All commit history becomes public.
- This cannot be easily reversed if sensitive information exists in history.

Understanding Repository Tabs and Features

- **Code:** Main file browser and repository content.
- **Issues:** Bug tracking and task management.
- **Pull Requests:** Code review and collaboration workflow.
- **Actions:** Automated workflows and continuous integration.
- **Projects:** Project management boards and planning tools.
- **Wiki:** Documentation and knowledge base.
- **Security:** Security alerts and vulnerability management.
- **Insights:** Analytics and repository statistics.
- **Settings:** Configuration and administration.

Academic Usage: Most educational use cases focus on Code, Issues, Projects, and Settings.

Collaboration and Access Management

Access Levels

- **Read:** Can view and download repository content.
- **Write:** Can view, download, and make changes.
- **Admin:** Full control including settings, deletion, and access management.

Collaboration Scenarios

- Teaching assistants may receive Write access to help manage course materials.
- Students typically receive Read access to view but not modify content.
- Research collaborators may receive Write access for active contribution.
- External reviewers might receive temporary Read access.

Other Important Settings Areas

- **Archive repository:** Make read-only while preserving content.
- **Transfer ownership:** Move repository to a different account or organization.
- **Delete repository:** Permanently remove repository and all content.
- **Template repository:** Allow others to create new repositories based on this one.

Adding Collaborators to Your Repository

1. Navigate to the repository's Settings tab.
2. Click “Collaborators” or “Manage access.”
3. Click “Add people” or “Invite a collaborator.”
4. Enter the GitHub username or email address.
5. Click “Add user to this repository.”

Invited users must explicitly accept the invitation to gain access.

Managing Collaborator Access

To remove collaborators:

1. Return to Settings → Collaborators or Manage access.
2. Find the collaborator you want to remove.
3. Click the settings or gear icon next to their name.
4. Select “Remove.”
5. Confirm the removal.

Academic Integrity and Repository Visibility

Critical Consideration for Students: Making academic work public can lead to academic misconduct violations. Always consult with instructors before changing repository visibility for coursework.

Common Academic Scenario

- Instructors provide repositories for assignments, exams, or projects.
- Students complete coursework in these repositories.
- After the course, students may want to showcase the work publicly.
- Making solutions public can enable academic dishonesty for future students.

Step 1: Always Ask Permission

Before making any coursework repository public:

- Contact your instructor directly.
- Explain your intention to use the work in your portfolio.
- Request explicit permission for public sharing.
- Get written confirmation when possible.

Step 2: Alternative Portfolio Strategies

Option A: Portfolio Repository

- Create a separate public repository for portfolio purposes.
- Write descriptions of your projects without sharing code.
- Include technologies used, challenges solved, and learning outcomes.

Option B: Controlled Access

- Keep coursework repositories private.
- Add potential employers as collaborators with read-only access.
- Provide temporary access for specific evaluation periods.
- Remove access after the evaluation process.

Option C: Selective Sharing

- Extract non-sensitive portions of your work.
- Create new repositories with generalized versions.
- Focus on showcasing skills rather than specific assignment solutions.

Professional Portfolio Development

- Summarize the problem you solved.
- List technologies and methodologies used.
- Highlight your specific contributions.
- Explain challenges overcome and lessons learned.

- Share representative code snippets only with permission.
- Create comprehensive README files.
- Contribute to open-source projects.
- Create personal projects that demonstrate similar skills.

Understanding Text Files

Text files store content in human-readable format using standard character encodings like ASCII or UTF-8. They contain only text characters without embedded formatting, images, or multimedia content.

Key Characteristics

- Human-readable when opened in any text editor.
- Content stored as plain characters.
- No embedded formatting, fonts, or multimedia.
- Changes can be easily identified line-by-line.
- Efficient for version control systems.

Common Text File Examples

- `.txt`: plain text files.
- `.md`: Markdown files.
- `.csv`: comma-separated values.
- Programming files: `.py`, `.java`, `.html`, `.css`, `.js`.
- Configuration files: `.json`, `.xml`, `.yaml`.
- Academic writing: `.tex`.

Understanding Binary Files

Binary files store data in machine-readable format that is not intended for direct human interpretation.

Key Characteristics

- Not human-readable in text editors.
- Machine-optimized storage format.
- Contains formatting, styling, or multimedia elements.
- Changes appear as incomprehensible binary differences.
- Inefficient for traditional version control.

Common Binary File Examples

- Microsoft Office: `.docx`, `.xlsx`, `.pptx`.
- PDFs: `.pdf`.
- Images: `.png`, `.jpg`, `.gif`.
- Audio and video: `.mp3`, `.mp4`, `.avi`.
- Compressed files: `.zip`, `.rar`.
- Executable files: `.exe`, `.app`.

Note: `.svg` files are usually text-based XML and can often be diffed by Git.

Why Git Struggles with Binary Files

Git was designed to track changes in source code, which is text-based. Git works best with text files because it can show meaningful line-by-line changes and merge them more effectively.

Binary File Limitations

- No meaningful diffs.
- Lower version-control value.
- No automatic merge capability.
- Large binary files slow down repository operations.

What Git Can Still Track

- Which files changed.
- Who made the changes.
- When changes were made.
- File size differences.

What Git Cannot Track Well

- What specifically changed inside the file.
- The nature or significance of changes.
- Content-based conflict resolution.

Appropriate Storage Strategies

Use GitHub and Git For

- Source code and programming files.
- Documentation such as Markdown and text files.
- Configuration files.
- Academic writing such as LaTeX files.
- Content where you need to track specific changes.
- Collaborative text-based projects.

Use Traditional Cloud Storage For

- Large multimedia files.
- Frequently changing binary files.
- Office documents such as Word, Excel, and PowerPoint.
- Images and graphics unless part of code projects.
- Content where you only need basic file sharing.

Examples of Good Storage Decisions

- Course materials: cloud storage for PowerPoint slides, GitHub for course code examples.
- Research projects: cloud storage for data files, GitHub for analysis scripts.
- Student projects: GitHub for code, cloud storage for presentations about the code.

Practical Considerations and Exceptions

Acceptable reasons to store binary files in Git:

- Convenience and organization.
- Small, static files such as logos or icons.
- Project requirements, such as game assets.
- Educational purposes.

Best practices when using binary files in Git:

- Keep files small when possible.
- Avoid frequent updates to binary files.
- Use clear commit messages explaining the purpose.
- Consider Git LFS for large binary assets.
- Document why binary files are included.

Understanding Organizations vs. Personal Accounts

Personal Account Structure

- URL format: <https://github.com/username>.
- Individual ownership of repositories.
- Basic collaboration through direct repository access.
- Best for personal projects and individual work.

Organization Structure

- URL format: <https://github.com/organization-name>.
- Shared ownership with multiple administrators.
- Team-based access control.
- Best for courses, research groups, and institutional projects.

Transferring Repositories to Organizations

1. Ensure you have an existing organization.
2. Verify you have administrative rights to the target organization.
3. Check that the repository name is available.
4. Navigate to the personal repository's Settings.
5. Scroll to the "Danger Zone."
6. Click "Transfer ownership."
7. Select the target organization.
8. Follow the multi-step verification process.

If you encounter an "insufficient collaborator seats" error, navigate to the organization's People tab, click "Manage seats," add additional seats, and retry the transfer.

Creating and Managing Teams

Step 1: Create Teams

In your organization, navigate to the "Teams" tab and click "New team."

Common academic teams include:

- Students.
- Assistants.
- Instructors.

Step 2: Configure Team Settings

- Use descriptive team names, such as `CS226-Students` or `CS226-TAs`.
- Keep teams visible for transparency.
- Add descriptions explaining each team's purpose.

Step 3: Invite Team Members

- Click “Add member” in each team.
- Enter GitHub usernames or email addresses.
- Track invitation status.

Configuring Repository Access Permissions

Permission Levels

- **Read:** View and download repository contents.
- **Triage:** Read access plus issue and pull request management.
- **Write:** Read and write access to repository contents.
- **Maintain:** Write access plus repository settings management.
- **Admin:** Full access including sensitive settings and deletion.

Academic Role Mapping

- **Students:** Read access for course materials.
- **Teaching Assistants:** Write access for updates and maintenance.
- **Instructors:** Admin access for full course management.

Advanced Organization Features

Nested Teams

Create sub-teams within larger teams. For example, a Students team may contain project-specific sub-teams.

Team Synchronization

Teams can connect to external identity providers and university directory systems in supported enterprise environments.

Organization-Level Settings

- Default permissions.
- Repository creation controls.
- Team creation controls.
- Security policies.

Academic Integration Considerations

- Organizations can handle large courses with hundreds of students.
- GitHub Classroom supports LMS integration for roster-related workflows in supported environments.
- Canvas integration can simplify roster import and identity mapping.
- Grades still need to be managed separately in Canvas or your institution's grading workflow.
- Single sign-on and enterprise access are separate from classroom roster integration.

Course Lifecycle Management

- **Semester Setup:** Create teams and set permissions.
- **Mid-Semester:** Add or remove students and adjust access.
- **End of Semester:** Archive or transfer repositories.
- **Multi-Semester:** Maintain organization across terms.

Privacy and Compliance

- Keep student work in private repositories.
- Control access to prevent unauthorized sharing.
- Plan for post-graduation access to student portfolios.

Troubleshooting Common Issues

- **Seat Limits:** Add more seats through organization billing. These may be free with education benefits.
- **Invitation Failures:** Check spam folders, provide direct organization URLs, and resend expired invitations.
- **Permission Confusion:** Organization owners automatically have admin access. Team permissions can override individual repository permissions.

Scenario Planning

Choose one scenario and design an optimal GitHub organization strategy.

Scenario A: Large Lecture Course

- Course materials repository.
- Assignment template repositories.
- Student submission repositories.
- TA collaboration space.

Scenario B: Research Seminar

- Shared reading materials.
- Individual research projects.
- Collaborative writing projects.
- Peer review workflows.

Scenario C: Multi-Section Course

- Shared curriculum materials.
- Section-specific resources.
- Cross-section collaboration.
- Administrative coordination.

For your chosen scenario:

- Plan repository organization and naming conventions.

- Define teams and their responsibilities.
- Map roles to access levels for each repository.
- Outline a semester timeline for setup and management.
- Identify where LMS integration would be valuable.

Chapter 4

Git Installation, Configuration, and Core Workflow

4.1 Operating System Compatibility

Git installation differs slightly depending on the operating system.

4.1.1 Pre-installed Systems

- **macOS:** Git is often already available, though first use may prompt installation of Xcode Command Line Tools.
- **Linux:** Git is commonly available through the system package manager but may still require manual installation.

4.1.2 Requires Installation

- **Windows:** Git must typically be downloaded and installed manually.

4.1.3 Verification

Open a command line interface and run:

```
git --version
```

If Git is installed correctly, you should see output similar to:

```
git version 2.41.0
```

4.2 Installing Git on Windows

4.2.1 Step 1: Download Git

1. Visit the official Git website:

`https://git-scm.com`

2. Navigate to “Downloads”
3. Select “Windows”
4. Download the installer

4.2.2 Step 2: Run the Installer

- Double-click the installer executable
- Accept the default installation directory
- Recommended options:
 - Desktop icon
 - Additional icons if desired

4.2.3 Step 3: Configure Installation Options

Recommended defaults:

- Text editor:
 - Notepad (recommended for beginners)
 - VS Code or Notepad++ if already installed
- OpenSSH configuration
- HTTPS backend
- Git Credential Manager
- Line ending conversions
- Terminal emulator

4.2.4 Step 4: Complete Installation

- Click “Install”
- Wait for installation to finish
- Launch Git Bash if desired

4.3 Accessing Git Interfaces

4.3.1 Windows

Git Bash

- Desktop shortcut
- Start menu search
- Right-click folder → “Git Bash Here”

Command Prompt

```
git --version
```

4.3.2 macOS

Open Terminal using:

- Cmd + Space
- Search “Terminal”

Verify installation:

```
git --version
```

4.3.3 Linux

Open a terminal and run:

```
git --version
```

Install Git using the system package manager if necessary.

4.4 Understanding Git Interfaces

4.4.1 Command Line Interfaces

- Git Bash (Windows)
- Terminal (macOS/Linux)
- Command Prompt (Windows)

4.4.2 Common Command Differences

Task	Git Bash / Terminal	Windows CMD
List files	<code>ls</code>	<code>dir</code>
Show directory	<code>pwd</code>	<code>cd</code>
Change directory	<code>cd</code>	<code>cd</code>

4.5 GUI Alternatives

Popular graphical Git tools include:

- GitHub Desktop
- GitKraken
- SourceTree
- VS Code Git integration

4.5.1 General Recommendation

- Learn command-line Git first
- Use GUI tools later for productivity

4.6 Basic Git Verification

4.6.1 Check Version

```
git --version
```

4.6.2 Explore Basic Commands

```
pwd
ls
git --help
```

Windows Command Prompt equivalent:

```
dir
```

4.7 Understanding Git Configuration

Git records:

- Author name
- Author email
- Commit timestamps

This metadata becomes part of the permanent project history.

4.8 Configuring User Identity

4.8.1 Configure Name

```
git config --global user.name "Your Full Name"
```

Example:

```
git config --global user.name "Jonathan Ma"
```

4.8.2 Configure Email

```
git config --global user.email "your.email@example.com"
```

4.8.3 Why Matching GitHub Email Matters

Matching your GitHub email provides:

- Commit attribution
- Contribution tracking
- GitHub profile linking

4.9 Understanding Configuration Scope

4.9.1 Global Configuration

Applies to all repositories:

```
git config --global
```

4.9.2 Repository-Specific Configuration

Applies only to the current repository:

```
git config
```

4.10 Verifying Configuration

4.10.1 View Name

```
git config user.name
```

4.10.2 View Email

```
git config user.email
```

4.10.3 View All Configuration

```
git config --list
```

4.11 Understanding Git Repositories

A Git repository contains:

- Project files
- Hidden `.git` metadata directory
- Version history
- Repository configuration

4.12 Creating a Repository

4.12.1 Create Project Directory

```
mkdir whack-a-box  
cd whack-a-box
```

4.12.2 Initialize Repository

```
git init
```

Expected output:

```
Initialized empty Git repository
```

4.13 Checking Repository Status

```
git status
```

Typical output:

```
On branch master
No commits yet
nothing to commit
```

4.14 The .git Directory

The hidden `.git` directory contains:

- Commit history
- Branch references
- Staging area
- Configuration

Important:

- Deleting `.git` removes all Git history
- The repository metadata lives entirely inside `.git`

4.15 Creating the First Project File

Create `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Whack-a-Box Game</title>
</head>
<body>
  <h1>Whack-a-Box Game</h1>
</body>
</html>
```


4.16 Understanding File States

Git categorizes files into states:

- Untracked
- Tracked
- Modified
- Staged
- Committed

Workflow progression:

Untracked → Tracked → Staged → Committed

4.17 Adding Files to Git

4.17.1 Check Status

```
git status
```

You should see:

```
Untracked files:
  index.html
```

4.17.2 Stage the File

```
git add index.html
```

Alternative:

```
git add .
```

4.17.3 Verify Staging

```
git status
```

Expected output:

```
Changes to be committed:
  new file:   index.html
```

4.18 Understanding the Staging Area

Git uses a three-stage workflow:

Working Directory → Staging Area → Repository

4.18.1 Why the Staging Area Exists

- Selective commits
- Review changes before committing
- Organize logical commit groups

4.19 Creating Commits

4.19.1 Commit Changes

```
git commit -m "Initial commit with basic HTML structure"
```

4.19.2 Verify Clean Status

```
git status
```

Expected:

```
nothing to commit, working tree clean
```

4.20 Viewing Commit History

4.20.1 Detailed Log

```
git log
```

4.20.2 Compact Log

```
git log --oneline
```

4.21 Making Additional Changes

Modify a tracked file, then check status:

```
git status
```

Expected output:

```
modified: index.html
```

Stage and commit again:

```
git add index.html
git commit -m "Update title formatting"
```

4.22 Commit Message Best Practices

Good commit messages:

- Use imperative mood
- Be descriptive
- Keep first line concise

Examples:

```
Add responsive navigation
Fix scoring bug
Update README documentation
```

4.23 The Three-Stage Architecture

Git operates with:

1. Working Directory
2. Staging Area
3. Repository History

Visualization:

Working Directory → Staging Area → Repository

4.24 Understanding `git diff`

View unstaged changes:

```
git diff
```

View staged changes:

```
git diff --staged
```

4.25 Creating a .gitignore

Create file:

```
touch .gitignore
```

Example contents:

```
.vscode/  
.DS_Store  
Thumbs.db  
*.log  
*.tmp
```

4.26 Why .gitignore Matters

Without .gitignore repositories become cluttered with:

- IDE configuration
- System files
- Temporary files
- Build artifacts

4.27 Common .gitignore Patterns

4.27.1 Python

```
__pycache__/  
*.pyc  
venv/
```

4.27.2 Node.js

```
node_modules/  
npm-debug.log*
```

4.27.3 LaTeX

```
*.aux  
*.bbl  
*.blg  
*.synctex.gz
```

4.28 Branching Concepts

Branches represent independent lines of development.

Benefits:

- Safe experimentation
- Parallel feature development
- Collaboration

4.29 Creating Branches

4.29.1 Create Branch

```
git branch space-theme
```

4.29.2 Switch Branch

```
git switch space-theme
```

Alternative:

```
git checkout space-theme
```

4.29.3 Create and Switch Simultaneously

```
git switch -c cyberpunk-theme
```

4.30 Branch Visualization

Useful command:

```
git log --oneline --graph --all
```

Conceptually:

```
HEAD -> main -> latest commit
```

4.31 Merging Branches

Merge feature branch into main branch:

```
git switch main  
git merge cyberpunk-theme
```

4.32 Deleting Branches

Safe delete:

```
git branch -d cyberpunk-theme
```

Force delete:

```
git branch -D experimental-branch
```

4.33 Understanding Git Stash

Stash temporarily stores unfinished work.

4.33.1 Create Stash

```
git stash
```

With message:

```
git stash push -m "Work in progress"
```

4.33.2 View Stashes

```
git stash list
```

4.33.3 Restore Stash

```
git stash pop
```

4.34 Local vs Remote Repositories

4.34.1 Local Repository

- Exists on your machine
- Full commit history
- Offline development

4.34.2 Remote Repository

- Hosted on GitHub
- Shared collaboration point
- Synchronization target

4.35 Connecting to GitHub

4.35.1 Add Remote

```
git remote add origin https://github.com/user/repo.git
```

4.35.2 Verify Remote

```
git remote -v
```

4.36 Pushing to GitHub

4.36.1 First Push

```
git push -u origin main
```

4.36.2 Subsequent Pushes

```
git push
```

4.37 Pulling Changes

4.37.1 Fetch Remote Updates

```
git fetch
```

4.37.2 Pull Remote Changes

```
git pull
```

4.37.3 Difference Between Fetch and Pull

- `git fetch`: download changes only
- `git pull`: fetch + merge

4.38 GitHub Pages

GitHub Pages provides free static website hosting.

4.38.1 Enable GitHub Pages

1. Open repository settings
2. Navigate to “Pages”
3. Select deployment branch
4. Save settings

4.38.2 Resulting URL

`https://username.github.io/repository-name/`

4.39 GitHub Desktop

GitHub Desktop provides a graphical Git interface.

Capabilities:

- Repository management
- Commit visualization
- Branch switching
- Push/pull synchronization

4.40 Recommended Beginner Workflow

Safe workflow:

```
git status
git add .
git commit -m "Descriptive message"
git push
```

Avoid using:

```
git reset --hard
git push --force
```

unless you fully understand the consequences.

4.41 Essential Git Commands Summary

Command	Purpose
<code>git status</code>	Check repository status
<code>git add .</code>	Stage changes
<code>git commit -m "..."</code>	Create commit
<code>git push</code>	Upload changes
<code>git pull</code>	Download changes
<code>git branch</code>	List branches
<code>git switch -c branch</code>	Create new branch
<code>git merge branch</code>	Merge branch
<code>git stash</code>	Temporarily save work
<code>git log -oneline</code>	View commit history

Chapter 5

GitHub Collaboration

5.1 What Are GitHub Issues?

5.1.1 Historical Context

GitHub Issues originally began as a bug tracking system.

In software development, a **bug** refers to an error or malfunction — something that does not behave as intended.

Bug tracking systems help teams:

- Document problems systematically
- Track debugging attempts
- Monitor issue resolution progress
- Assign responsibility
- Organize release planning

5.1.2 Modern Usage

GitHub Issues evolved beyond simple bug tracking and are now used for:

- Feature planning
- Task management
- Team discussion
- Project organization
- Research coordination

5.1.3 Community Adoption

GitHub Issues became a central communication system within open source software communities.

Contributors use issues to discuss:

- Bugs and fixes
- Feature ideas
- Documentation improvements
- Design decisions
- Architectural planning

5.2 Best Practices for Writing Issues

5.2.1 Software Projects

Strong issue reports should include:

- Expected behavior
- Actual behavior
- Steps to reproduce the issue
- Relevant screenshots or logs
- System information when applicable

5.2.2 Non-Software Projects

For research, writing, or planning projects:

- Clearly describe the task
- Explain why the task matters
- Include deadlines or dependencies
- Provide sufficient context

5.3 Linking to Specific Code or Content

5.3.1 Creating Line References

To reference code directly:

1. Open the relevant file in GitHub
2. Click the desired line number
3. Copy the generated URL
4. Paste the URL into the issue description

5.3.2 Reference Formats

Examples:

```
filename.js:L25  
filename.js:L25-L35
```

5.3.3 Permalinks

GitHub also supports permanent references:

- Right click line selection
- Choose “Copy permalink”

Permalinks remain stable even if the file changes later.

5.4 Issues Beyond Software Development

5.4.1 Academic Applications

GitHub Issues work well for:

- Manuscript collaboration
- Research task management
- Literature review coordination
- Course development

5.4.2 Research Examples

Possible issue examples:

- “Revise methodology section”
- “Collect additional survey responses”
- “Update bibliography formatting”
- “Analyze experimental results”

5.5 Issue Categories

5.5.1 Bugs

Problems with existing functionality.

Example:

`Filter buttons are not working`

5.5.2 Features and Enhancements

New functionality to be added.

Examples:

`Add dark mode toggle`

`Add to-do counter`

`Add export functionality`

5.5.3 General Tasks

Not all issues are bugs or features.

Examples include:

- Documentation updates
- Writing tasks
- Research activities
- Administrative work

5.6 Applying Labels to Issues

5.6.1 Accessing Labels

1. Open an issue
2. Locate the “Labels” section
3. Click the gear icon

5.6.2 Common Labels

Typical labels include:

- bug
- enhancement
- documentation
- question

5.6.3 Viewing Labels

Labels appear directly in the issues list and use color coding for quick identification.

5.7 Filtering Issues

5.7.1 Label-Based Filtering

GitHub allows filtering by label.

Examples:

- View only bugs
- View only enhancements
- Filter documentation tasks

5.7.2 Organizational Benefits

Filtering supports:

- Focused work sessions
- Prioritization
- Team specialization
- Progress tracking

5.8 Customizing Labels

5.8.1 Managing Labels

Navigate to the “Labels” tab in the repository.

Capabilities include:

- Rename labels
- Change colors
- Delete unused labels
- Create custom labels

5.8.2 Academic Label Examples

- Research
- Writing
- Review
- Data Collection
- Analysis

5.8.3 Software Label Examples

- Bug
- Feature
- Performance
- Documentation
- Testing

5.9 Advanced Label Usage

5.9.1 Duplicate Issues

Large projects often encounter repeated reports.

Strategies:

- Apply a `duplicate` label
- Reference the original issue

5.9.2 Won't Fix

The `won't fix` label documents intentional non-implementation decisions.

5.9.3 Good First Issue

Common in open source projects.

Purpose:

- Identify beginner-friendly tasks
- Encourage community participation

5.10 Assigning Issues

5.10.1 Assigning Team Members

To assign an issue:

1. Open the issue
2. Locate “Assignees”
3. Select collaborators

5.10.2 Benefits of Assignment

Assignments provide:

- Accountability
- Workload visibility
- Responsibility tracking
- Team coordination

5.11 Multiple Assignees

GitHub now supports multiple assignees for collaborative work.

Useful for:

- Paired development
- Mentorship
- Review coordination
- Complex interdisciplinary tasks

5.12 Academic Applications of Assignment

5.12.1 Research Teams

Assignments help coordinate:

- Manuscript sections
- Literature reviews
- Data collection
- Statistical analysis

5.12.2 Course Development

Assignments help distribute:

- Lecture creation
- Assessment design
- Technology integration
- Review cycles

5.13 GitHub Flow Branch Strategy

5.13.1 Why Use Branches?

Directly editing the main branch creates risks.

Dedicated branches provide:

- Isolation
- Safe experimentation
- Parallel development
- Easier review workflows

5.13.2 GitHub Flow Process

Standard workflow:

Branch Creation → Development → Pull Request → Review → Merge

5.14 Creating Branches

5.14.1 Check Current Branch

```
git branch
```

5.14.2 Create and Switch Branch

```
git checkout -b branch-name
```

Modern alternative:

```
git switch -c branch-name
```

5.14.3 Verify Branch

```
git branch
```

5.15 Branch Naming Conventions

5.15.1 Common Naming Patterns

Bug fixes:

```
bugfix/filter-buttons  
fix/navigation-error
```

Features:

```
feature/dark-mode  
feat/user-authentication
```

5.15.2 Including Issue Numbers

Examples:

```
bugfix/issue-123  
feature/issue-456
```

5.16 Working in Branches

5.16.1 Development Process

Typical workflow:

```
git add .  
git commit -m "Fix filter buttons"  
git push -u origin branch-name
```

5.16.2 Verification

Confirm the branch appears on GitHub with recent commits.

5.17 Testing and Validation

Before creating a pull request:

- Verify functionality
- Test edge cases
- Ensure no regressions occurred

5.18 Understanding Pull Requests

5.18.1 Definition

A pull request asks:

“I made some changes. Can these changes be reviewed and merged?”

5.18.2 Why Pull Requests Matter

Pull requests provide:

- Code review
- Team discussion
- Documentation
- Quality assurance

5.19 Creating Pull Requests

5.19.1 Basic Workflow

1. Open repository
2. Navigate to “Pull requests”
3. Click “New pull request”
4. Select branches
5. Review changes

5.19.2 Branch Selection

- Base branch: usually `main`
- Compare branch: feature branch

5.20 Writing Pull Request Descriptions

Good pull request descriptions include:

- Summary of changes
- Testing performed
- Related issues
- Implementation details

5.20.1 Issue References

Reference issues using:

#1
#25
#104

GitHub automatically creates links.

5.21 Code Review

5.21.1 Purpose of Review

Code review improves:

- Quality
- Readability
- Consistency
- Collaboration

5.21.2 Review Process

Reviewers can:

- Approve changes
- Request changes
- Leave comments

5.22 Pull Request Interface

Main tabs include:

- Conversation
- Commits
- Files Changed

5.22.1 Files Changed

Visual diffs display:

- Red lines: deletions
- Green lines: additions

5.23 Task Lists in Issues

5.23.1 Markdown Syntax

- [] Incomplete task
- [x] Completed task

5.23.2 Interactive Features

GitHub converts Markdown checklists into interactive checkboxes.

5.24 Sub-Issues

Task lists can become sub-issues.

Benefits:

- Hierarchical organization
- Parallel task management
- Progress tracking

5.25 Progress Tracking

Parent issues display:

- Percentage completion
- Sub-issue counts
- Visual progress bars

5.26 GitHub Codespaces

5.26.1 Definition

GitHub Codespaces provides a cloud-hosted development environment. Essentially:

Visual Studio Code running in the browser.

5.26.2 Capabilities

Codespaces supports:

- Integrated terminals
- Git operations
- Extensions
- Browser-based development

5.27 Launching Codespaces

5.27.1 Workflow

1. Open repository
2. Click “Code”
3. Select “Create codespace”

5.27.2 Environment Setup

GitHub automatically:

- Creates cloud resources
- Clones the repository
- Opens VS Code in-browser

5.28 Branching in Codespaces

Create branches normally using terminal commands:

```
git checkout -b feature/todo-counter
```

5.29 Codespaces Advantages

- No local setup required
- Consistent environments
- Accessible from any computer
- Integrated GitHub workflow

5.30 Codespaces Considerations

- Requires internet connection
- Usage quotas may apply
- Resource limitations exist

5.31 Reviewing Pull Requests

5.31.1 Review Workflow

1. Open pull request
2. Examine changes
3. Leave comments
4. Approve or request changes

5.31.2 Review Types

Possible outcomes:

- Approve
- Request Changes
- Comment Only

5.32 Line-Level Feedback

GitHub supports comments on specific lines.

Benefits:

- Precise communication
- Contextual discussion
- Easier collaboration

5.33 AI-Assisted Review

Modern tools such as GitHub Copilot assist with:

- Code suggestions
- Style corrections
- Error detection
- Documentation improvements

5.34 Updating Pull Requests

After feedback:

```
git add .  
git commit -m "Address PR feedback"  
git push origin feature-branch
```

GitHub automatically updates the pull request.

5.35 Resolving Conversations

After implementing changes:

- Reply to reviewer comments
- Resolve discussion threads
- Request re-review

5.36 Approving Pull Requests

Common approval phrase:

LGTM

Meaning:

Looks Good To Me

5.37 Merging Pull Requests

5.37.1 Merge Process

1. Verify approvals
2. Click “Merge pull request”
3. Confirm merge

5.37.2 Post-Merge Actions

After merging:

- Close related issues
- Delete old branches
- Verify functionality

5.38 Merge Conflicts

5.38.1 What Causes Conflicts?

Conflicts occur when:

- Multiple contributors edit the same lines
- File structures change simultaneously
- Branches diverge significantly

5.38.2 Conflict Indicators

GitHub displays warnings such as:

“This branch has conflicts that must be resolved”

5.39 Conflict Resolution

5.39.1 Conflict Markers

Git inserts markers such as:

```
<<<<<<< HEAD
=====
>>>>>>> feature-branch
```

5.39.2 Resolution Steps

1. Edit the conflicted section
2. Remove markers
3. Test functionality
4. Commit the resolution

5.40 Preventing Conflicts

Strategies include:

- Frequent synchronization
- Smaller feature branches
- Team communication
- Modular project structure

5.41 Understanding Milestones

5.41.1 Purpose

Milestones group issues toward common project goals.

Examples:

- Version 1.0 Release
- Manuscript First Draft
- Research Phase Completion

5.42 Creating Milestones

1. Open Issues tab
2. Select “Milestones”
3. Create milestone
4. Add title and due date

5.43 Milestone Progress

GitHub automatically tracks:

- Completion percentages
- Closed issues
- Remaining tasks

5.44 Project Boards

5.44.1 Kanban Workflow

Project boards visualize workflow stages.

Typical columns:

- Backlog
- In Progress
- Review
- Done

5.45 Board Automation

GitHub supports automation such as:

- Move assigned issues to “In Progress”
- Move merged pull requests to “Done”

5.46 Discussion Forums

5.46.1 Discussions vs. Issues

Issues:

- Action-oriented
- Task-based

Discussions:

- Brainstorming
- Open-ended communication
- Community interaction

5.47 Discussion Categories

Common categories:

- General
- Ideas
- Q&A
- Announcements

5.48 GitHub Wikis

5.48.1 Purpose

GitHub Wikis support:

- Detailed documentation
- Tutorials
- Team knowledge bases
- Technical references

5.48.2 README vs. Wiki

README files provide quick overviews.

Wikis provide long-form documentation.

5.49 Wiki Organization

Typical wiki structure:

- Home Page
- Getting Started
- User Guide
- FAQ
- Changelog

5.50 Issue Templates

5.50.1 Benefits

Templates improve:

- Consistency
- Information quality
- Team efficiency

5.51 Bug Report Template Example

Problem Description

Steps to Reproduce

Expected Behavior

Actual Behavior

Environment

5.52 Feature Request Template Example

Feature Description

Use Case

Proposed Solution

Alternatives Considered

5.53 Markdown vs YAML Templates

5.53.1 Markdown Templates

- Simple to create
- Flexible
- Stored as .md files

5.53.2 YAML Forms

- Structured input
- Required fields
- Cleaner issue submission

5.54 Fork-Based Collaboration

5.54.1 Why Forks Exist

Open source projects cannot practically add every contributor as a collaborator.

Forks allow:

- Independent contribution
- Safe experimentation
- Scalable collaboration

5.55 Creating Forks

1. Open repository
2. Click “Fork”
3. Choose destination account

5.55.1 Fork Relationship

GitHub tracks the parent-child relationship automatically.

5.56 Fork Workflow

5.56.1 Contribution Process

Fork → Clone → Branch → Develop → Pull Request

5.57 Cross-Repository Pull Requests

Fork-based pull requests target:

- Source: contributor fork
- Target: original repository

5.58 Open Source Contribution Etiquette

Good practices include:

- Read contributing guidelines
- Respect maintainer decisions
- Provide high-quality work
- Respond professionally to feedback

5.59 Complete GitHub Collaboration Workflow

Full collaboration cycle:

Issues → Assignment → Branching → Development → Pull Request → Review → Merge → Project Track

5.60 Collaboration Beyond Software

These workflows apply equally to:

- Academic research
- Manuscript collaboration
- Course development
- Team writing projects
- Organizational planning

GitHub provides a unified framework for collaborative project management across both technical and non-technical disciplines.

Chapter 6

GitHub Education

6.1 What is GitHub Classroom?

6.1.1 Platform Access

URL:

- `classroom.github.com`

Authentication:

- Sign in using an existing GitHub account

Integration:

- Seamless connection with GitHub's repository system

6.1.2 Core Problem Solved

Traditional approaches to distributing assignment repositories require:

- Manual repository creation for each student or team
- Individual access permission setup for every repository
- Starter code distribution through email or file sharing
- Complex coordination between instructors and students

GitHub Classroom Solution:

- Automated repository provisioning based on assignment templates
- Invite link distribution that triggers repository creation
- Standardized access control across all student repositories
- Centralized assignment management through classroom dashboard

6.2 Alternative Approaches

6.2.1 Manual Repository Management

- Instructors create repositories individually for each student
- Time-intensive for large classes
- Prone to configuration errors and inconsistencies

6.2.2 Student Self-Setup

- Provide guidelines for students to create their own repositories
- Requires additional instruction and troubleshooting
- Inconsistent repository naming and structure

6.2.3 API-Based Automation

- Use GitHub's API to programmatically create repositories
- Requires scripting skills and technical expertise
- Custom solution development and maintenance

6.2.4 GitHub Classroom Advantages

- No coding required — web-based interface for all operations
- Standardized workflow — consistent experience across assignments
- Built-in features — roster management, grading integration, feedback tools

6.3 Platform Evolution and Capabilities

6.3.1 Origin and Development

- Side project initiated by GitHub employees
- Educational focus addressing instructor pain points
- Continuous expansion of features and integrations

6.3.2 Current Feature Set

- Repository provisioning from assignment templates
- Roster management for student enrollment tracking
- Limited grading capabilities and feedback mechanisms
- Integration potential with learning management systems

6.3.3 Future Considerations

- Expanding functionality as the platform matures
- Enhanced grading features and assessment tools
- Deeper LMS integration for seamless workflow

6.4 Organization Requirements and Relationships

6.4.1 Required Setup

- One organization per classroom
- Multiple classrooms per organization
- Organizational flexibility depending on instructor preferences

6.4.2 Organization Strategies

Semester-Specific Organizations

- Create a new organization each semester
- Example: CS226-FA25
- Benefits:
 - Clean separation
 - Archived course materials
 - Fresh start each term

Course-Persistent Organizations

- Use same organization across multiple semesters
- Benefits:
 - Centralized management
 - Consistent structure
 - Historical access

Specialized Organizations

- Separate organizations for different purposes
- Example: CS226-FA25-homework

6.5 Creating Educational Organizations

6.5.1 Step 1: Organization Creation

1. Navigate to GitHub organization creation
2. Choose “Create a free organization”
3. Enter organization name
4. Example: `classroom-demo-schoolu`
5. Accept terms and complete setup

6.5.2 Naming Convention Examples

Course Code Format

- CS226-FA25
- EN-601-226-SP23

Institution-Specific Format

- `schoolu-cs226-fall2025`
- `school-data-science-2023`

Purpose-Specific Format

- `cs226-homework-fa22`
- `data-structures-labs-2023`

Faculty can request an Organization from the OSPO. If WSE faculty plan to connect Classroom to Canvas, being part of the Enterprise is required.

6.6 Organization Configuration Considerations

6.6.1 Free vs. Paid Organization

- Free organizations sufficient for basic functionality
- Team/Enterprise features needed for private repositories
- Upgrade timing can occur later

6.6.2 Funding Options

- GitHub Education benefits
- Institutional Enterprise accounts
- Personal payment for small courses

6.6.3 Privacy and Access Control

- Public repositories work with free organizations
- Private repositories require paid features
- Student privacy may require private repositories

6.7 Classroom Creation Process

6.7.1 Step 1: Access Classroom Creation

1. Navigate to `classroom.github.com`
2. Sign in
3. Click “New classroom”

6.7.2 Step 2: Organization Selection

- Select target GitHub organization
- Example: classroom-demo-schoolu

6.7.3 Step 3: Classroom Naming

Auto-generated name example:

- classroom-demo-schoolu-classroom-abc123

Customization:

- Remove auto-generated suffix
- Use cleaner course-based naming

6.8 Initial Configuration Options

6.8.1 Teaching Staff Configuration

- Invite TAs and admins
- Can be skipped initially
- Configure later if desired

6.8.2 Student Roster Management

- LMS integration
- Manual student entry
- CSV upload
- Deferred setup possible

6.9 Classroom Interface Overview

6.9.1 Main Interface Tabs

Settings Tab

- Classroom name
- Administrative configuration
- Integration settings

People Management

- TAs and Admins
- Students
- Role-based permissions

Assignments Tab

- Core assignment management functionality
- Repository provisioning controls
- Student assignment distribution

6.10 Repository Creation Requirements

6.10.1 Organizational Requirement

- Template repositories must exist within the same organization
- Ensures proper permissions and visibility

6.10.2 Repository Visibility

- Private repositories recommended for educational content
- Public repositories optional for open coursework

6.11 Template Repository Setup

6.11.1 Step 1: Repository Creation

1. Navigate to organization repositories
2. Click “New repository”
3. Name repository descriptively
4. Example: `homework-1`
5. Configure privacy settings

6.11.2 Step 2: Add Assignment Content

Example structure:

```
homework-1/  
+--README.md  
+-- starter-code.py  
+-- data/  
+-- requirements.txt
```

6.11.3 Step 3: Configure Template Settings

1. Navigate to repository Settings
2. Locate “Template repository”
3. Enable template option

6.12 Template Repository Features

Definition:

GitHub template repositories allow generation of new repositories with identical structure and content.

6.12.1 Key Benefits

- Standardized starting point
- Consistent assignment organization
- Reduced setup time
- Instructor control over materials

6.13 Assignment Design Considerations

6.13.1 Assignment Instructions

- Comprehensive README
- Step-by-step requirements
- Submission guidelines
- Grading rubrics

6.13.2 Starter Materials

Programming Assignments

- Skeleton code
- Test files
- Configuration files
- Sample datasets

Writing Assignments

- Document templates
- Citation guidelines
- Formatting instructions

6.14 Assignment Creation Workflow

6.14.1 Step 1: Access Assignment Creation

1. Navigate to GitHub Classroom
2. Select classroom
3. Click “Create an assignment”

6.14.2 Step 2: Basic Assignment Configuration

Assignment Title

- Example: “Homework 1”
- Spaces converted to dashes automatically

Deadline Configuration

- Optional
- Can enforce write-access cutoff

6.14.3 Step 3: Assignment Type Selection

- Individual assignments
- Group assignments

6.15 Repository Configuration Options

6.15.1 Template Repository Selection

- Search by repository name
- Repository must be marked as template

6.15.2 Student Repository Visibility

Private Repositories

- Recommended default
- Protects student privacy
- Supports academic integrity

Public Repositories

- Useful for portfolio projects
- Encourages peer collaboration

6.15.3 Access Control Settings

- Standard write access
- Optional admin access

6.16 Advanced Assignment Options

6.16.1 Branch Management

Default Branch Only

- Standard clean starting point

All Branches

- Includes all branches from template
- Useful for phased assignments

6.16.2 Specialized Features

- GitHub Codespaces integration
- Protected file paths
- Auto-grading tests
- Pull request feedback

6.17 Assignment Dashboard and Management

6.17.1 Assignment Dashboard Features

- Student participation tracking
- Invite link management
- Submission monitoring

6.17.2 Distributing Assignment Invites

- LMS posting
- Email distribution
- Course website announcements

6.18 Student Acceptance Workflow

6.18.1 Step 1: Assignment Access

- Student clicks invite link
- Authenticates with GitHub

6.18.2 Step 2: Repository Naming Convention

- Format:

`[assignment-name] - [username]`

Example:

`homework-1-ali-the-student`

6.18.3 Step 3: Assignment Acceptance

- Click “Accept this assignment”
- Repository automatically provisioned

6.19 Repository Organization and Permissions

6.19.1 Repository Characteristics

- Organization-hosted repositories
- Standardized naming conventions
- Centralized instructor oversight

6.19.2 Student Permissions

- Write access
- Limited settings access

6.19.3 Instructor Permissions

- Full administrative control
- Assignment monitoring
- Access management

6.20 Classroom Dashboard Monitoring

6.20.1 Assignment Tracking Features

- Acceptance status
- Latest commits
- Submission timestamps

6.20.2 Administrative Tools

- Individual deadline extensions
- Accommodation support

6.21 Student Workflow Overview

6.21.1 Development Process

- Clone repository locally
- Complete assignment work
- Commit changes
- Push to repository

6.21.2 Submission Recognition

- Any pushed commit counts as submission activity
- Dashboard updates automatically

6.22 Practical Submission Example

6.22.1 Step 1: Assignment Modification

- Edit assignment files
- Replace placeholders with completed work

6.22.2 Step 2: Committing and Submission

```
git add .  
git commit -m "Complete assignment"  
git push
```

6.22.3 Step 3: Submission Confirmation

- Instructor dashboard updates automatically

6.23 Submission Tracking Limitations

6.23.1 Push-Equals-Submission Model

- Every push considered submission activity
- Frequent commits may create ambiguity

6.23.2 Potential Issues

- Work-in-progress confusion
- Final version uncertainty
- Student misunderstanding of submission timing

6.24 Alternative Submission Strategies

6.24.1 Instructor Communication

- Define final submission expectations clearly
- Use commit message conventions

6.24.2 Workflow Adaptations

- Explicit submission checkpoints
- Dedicated submission branches

6.25 Gradescope Assignment Configuration

6.25.1 Step 1: Create Programming Assignment

- GitHub integration requires “Programming Assignment”

6.25.2 Step 2: Enable GitHub Submission Method

- Enable GitHub integration in assignment settings

6.26 GitHub Account Integration

6.26.1 Instructor Setup

- Link GitHub account through Gradescope settings

6.26.2 Student Setup

- Connect GitHub account during first submission

6.27 Organizational Access Configuration

6.27.1 Critical Requirement

Students cannot access organization repositories through Gradescope until organizational access is granted.

6.27.2 Configuration Steps

1. Open GitHub Settings
2. Navigate to Applications
3. Open Authorized OAuth Apps
4. Grant Gradescope access to organization

6.28 Student Submission Workflow

6.28.1 Repository Selection

- Search for assignment repository
- Select appropriate branch

6.28.2 Submission

- Click “Upload”
- Verify repository contents

6.29 Workflow Limitations and Considerations

6.29.1 Manual Submission Requirement

- Repository updates do not automatically sync to Gradescope
- Students must manually resubmit

6.29.2 Integration Benefits

- GitHub development workflow
- Gradescope grading infrastructure
- Repository history preservation